Undirected **TSP** as a Constrained **GSTP** Variant

Yılmaz Arslanoğlu

Independent Researcher

Traveling Salesman Problem (**TSP**)

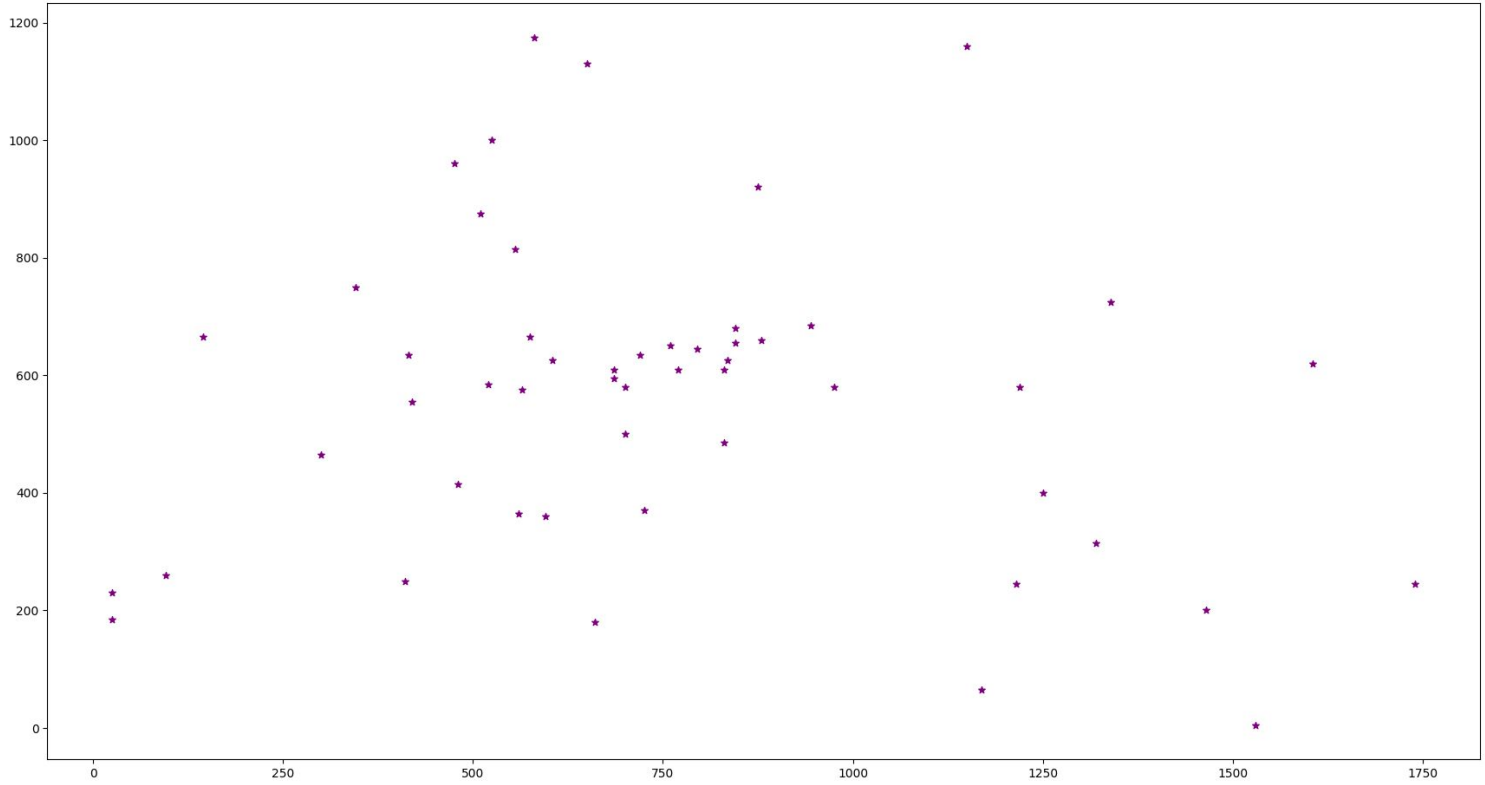
Given

- a list of **cities**, and
- the distance between every pair,

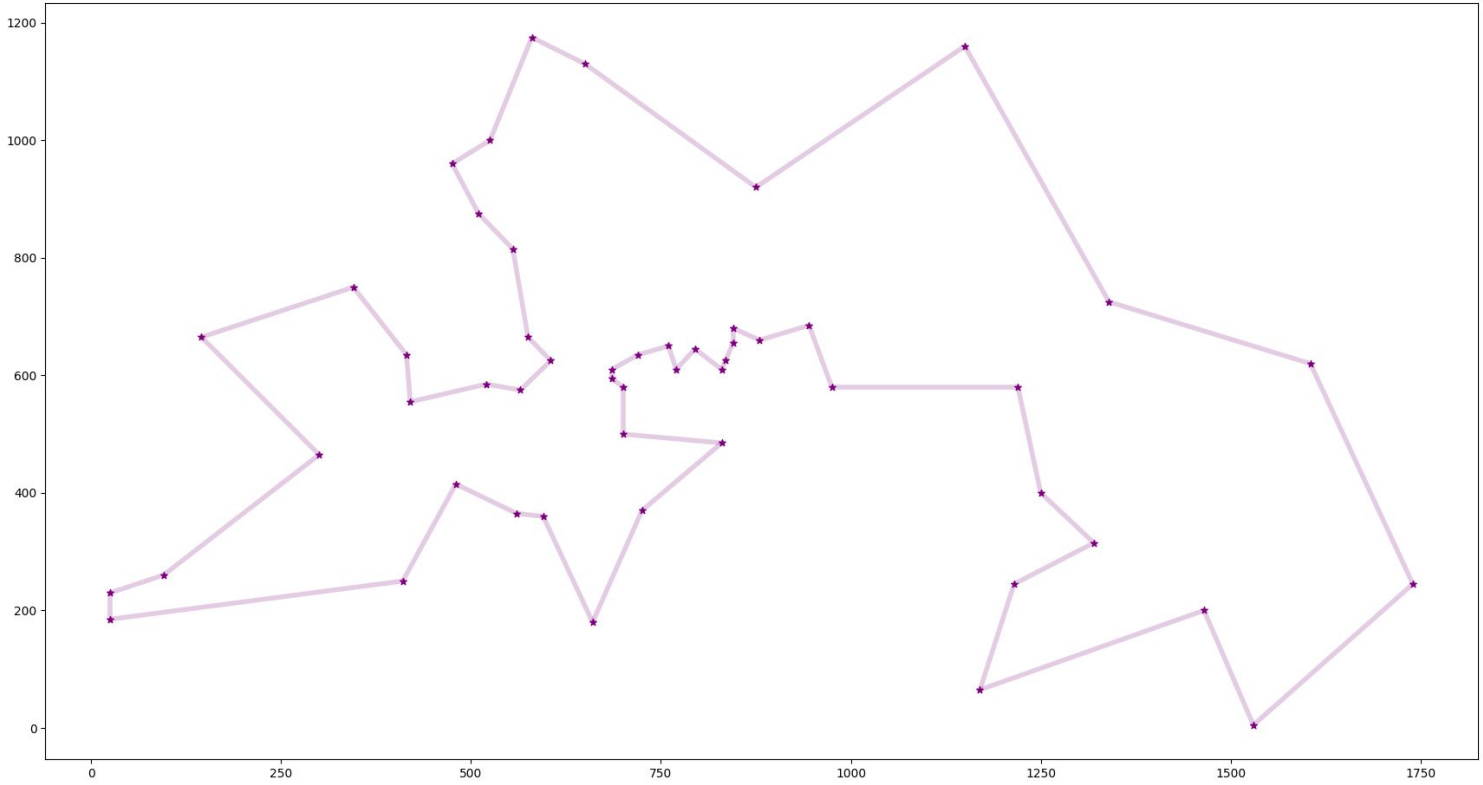
Find the **shortest** possible **route** that

- visits each city <u>exactly once</u>, and
- <u>returns back</u> to the starting city.

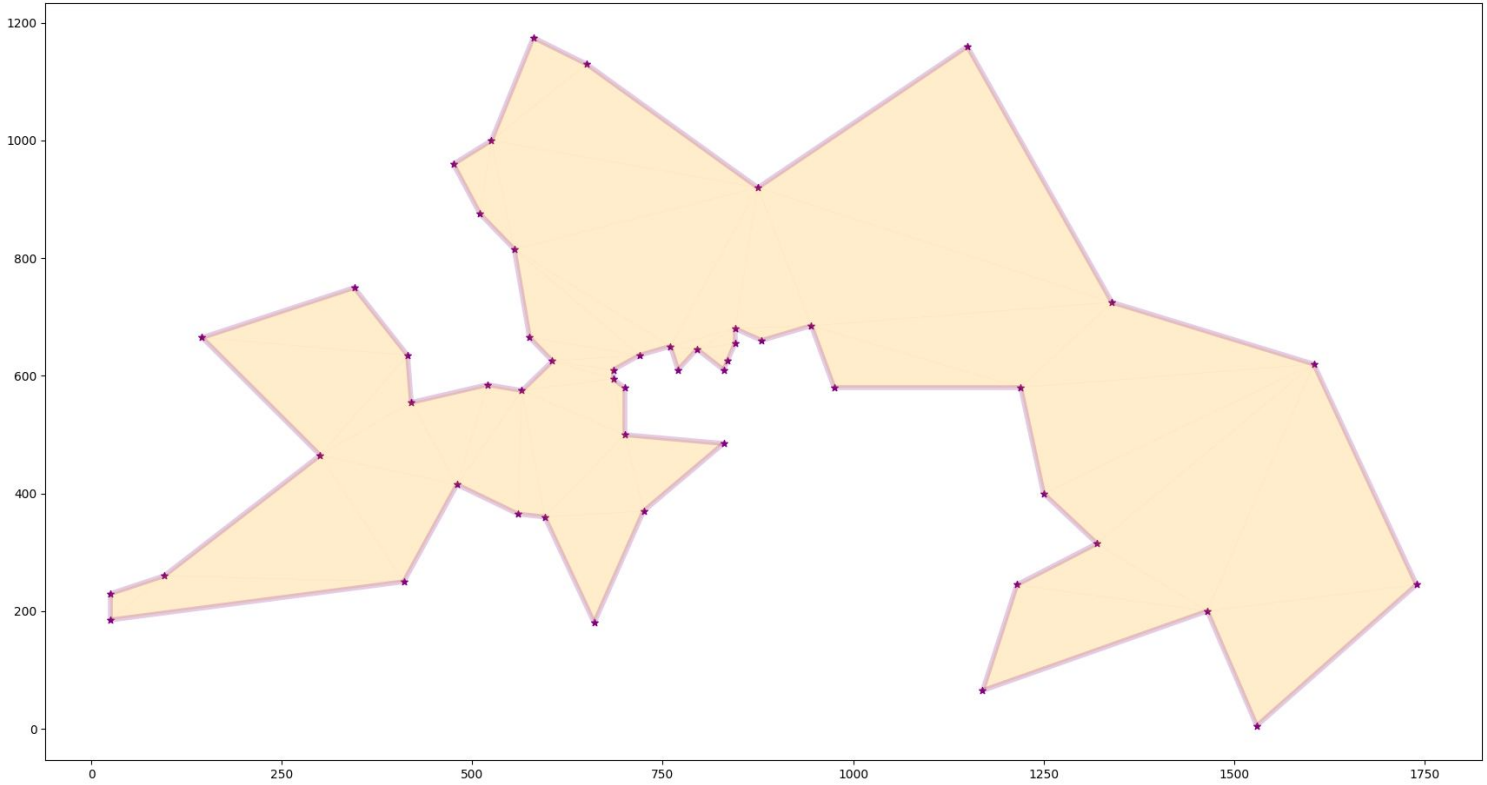
berlin52 (from TSPLIB)



Optimal Tour



A **Surface** is enclosed (Boundary → Surface)



Perspective Shift ~~(Boundary → Surface)~~
(Surface → Boundary)

find the shortest **1D Cycle** ,
given edges between city-pairs

find the **2D Surface** ,
(whose **induced boundary** is the shortest)

given ...

The Input: A **Set of Triangles**
(*every 3-city defines a triangle*)

$$\mathcal{T} \subseteq \binom{V}{3}$$

Candidate Triangle Set: The **Trade-off**

{empty set}

$\emptyset$

infeasible ❌

$\mathcal{T}_{Delaunay}$

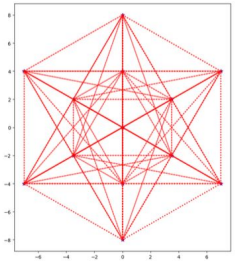
efficient heuristic ✅
(feasibility not guaranteed)

{universal set}

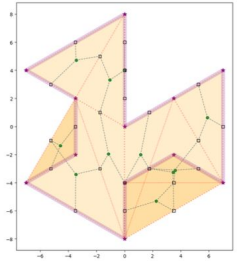
$\mathcal{K}_{All} := \binom{V}{3}$

exact optimal 🎯
(but intractable ♾️)

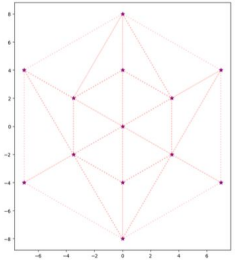
Universal Set vs. Delaunay **Triangulation**



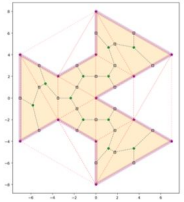
(a) Dense Input ($\mathcal{K}_{All}$)



(b) Dense Solution (non-planar; overlap is a projection artifact)

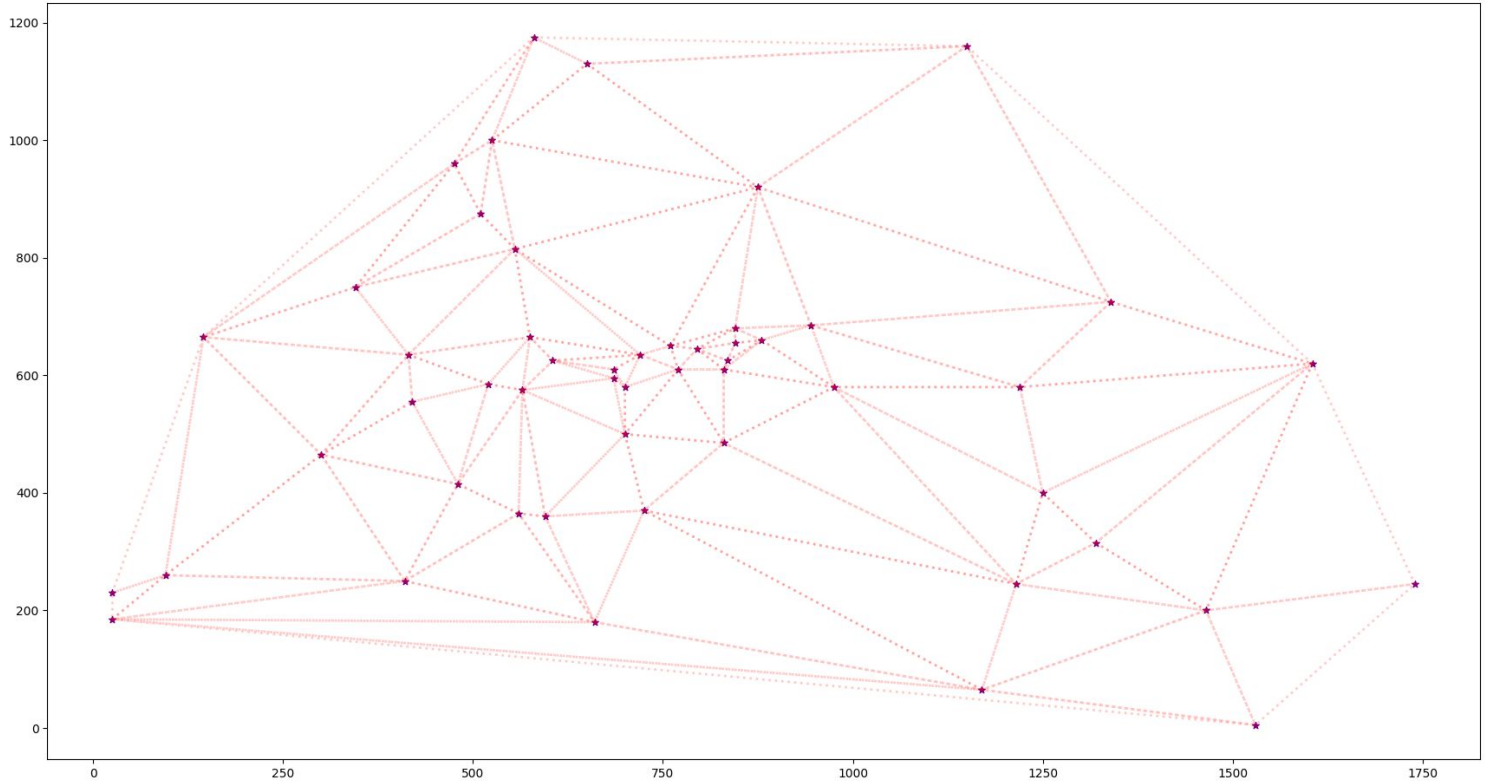


(c) Delaunay Input ($\mathcal{T}_{Delaunay}$)

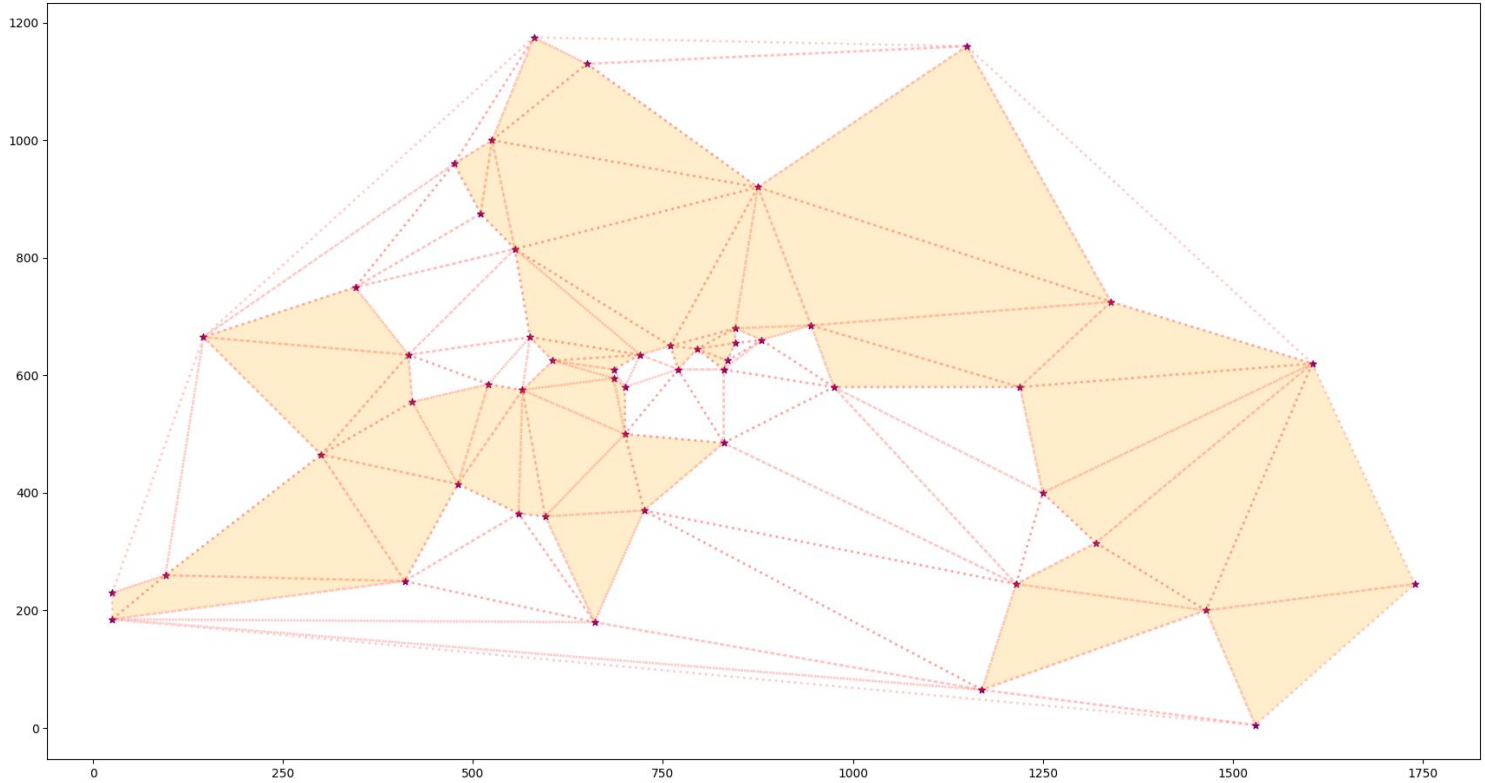


(d) Delaunay Solution (Planar)

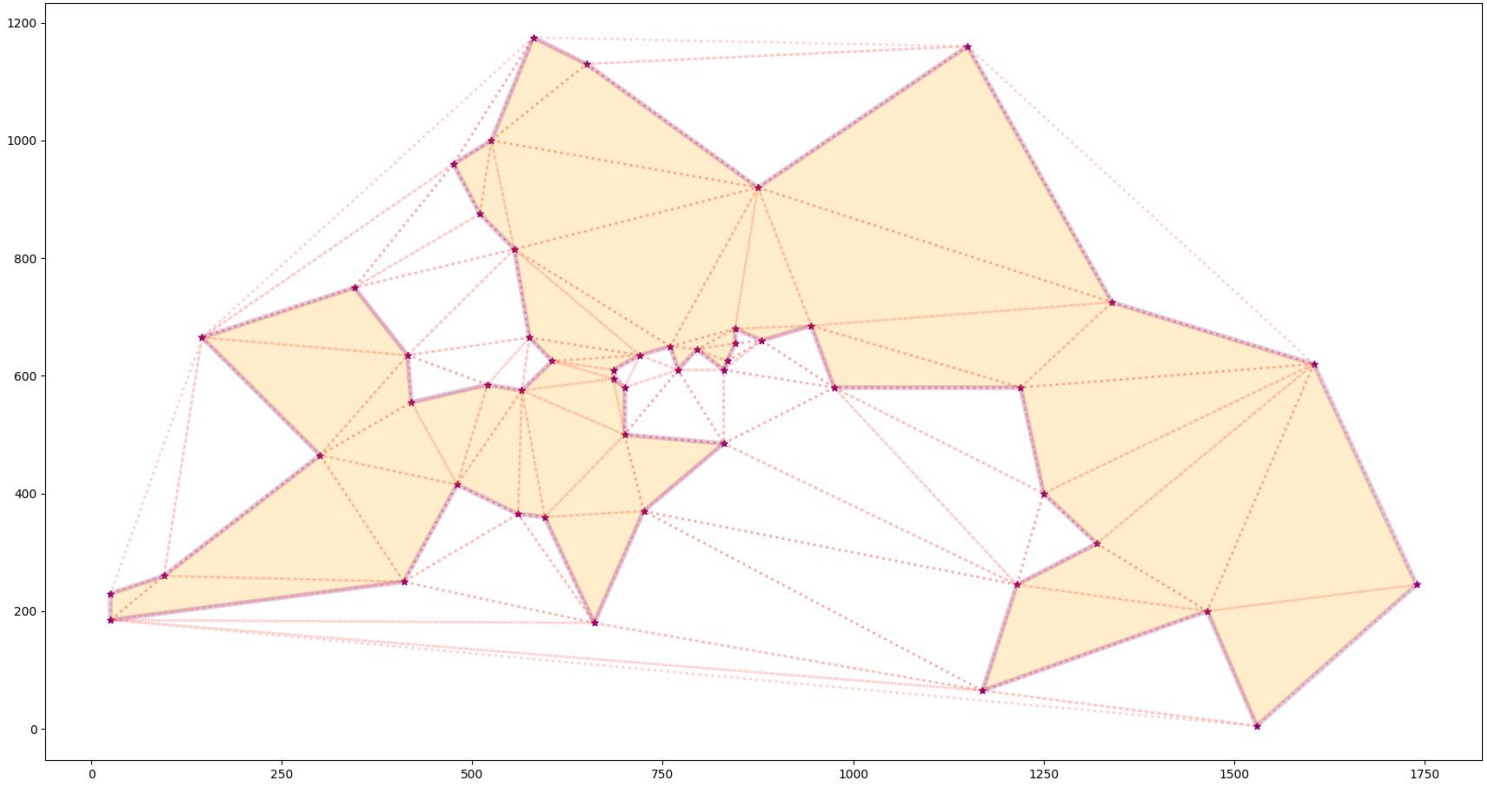
Delaunay Triangulation of **berlin52**



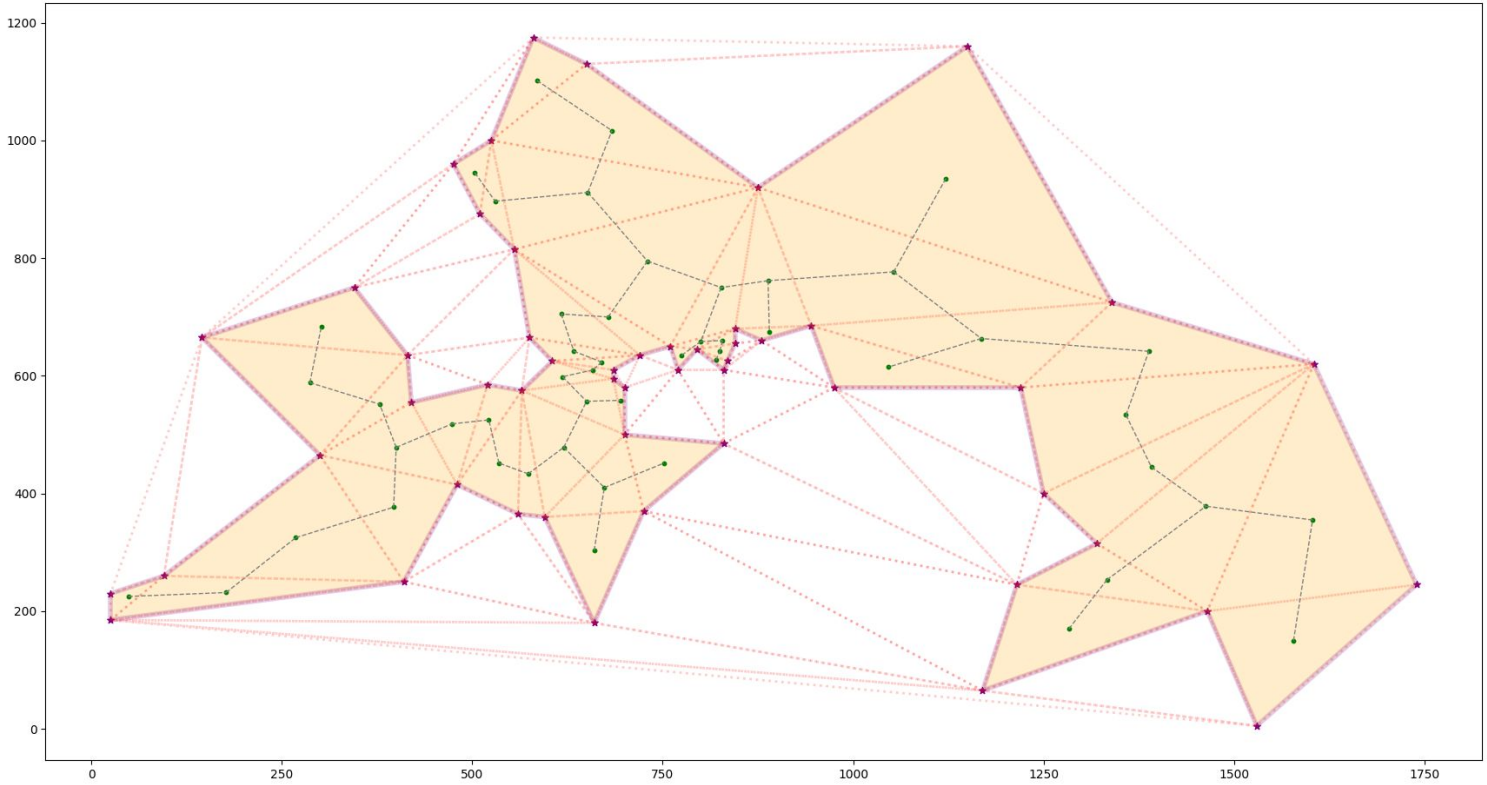
Selected Triangles (the building blocks)



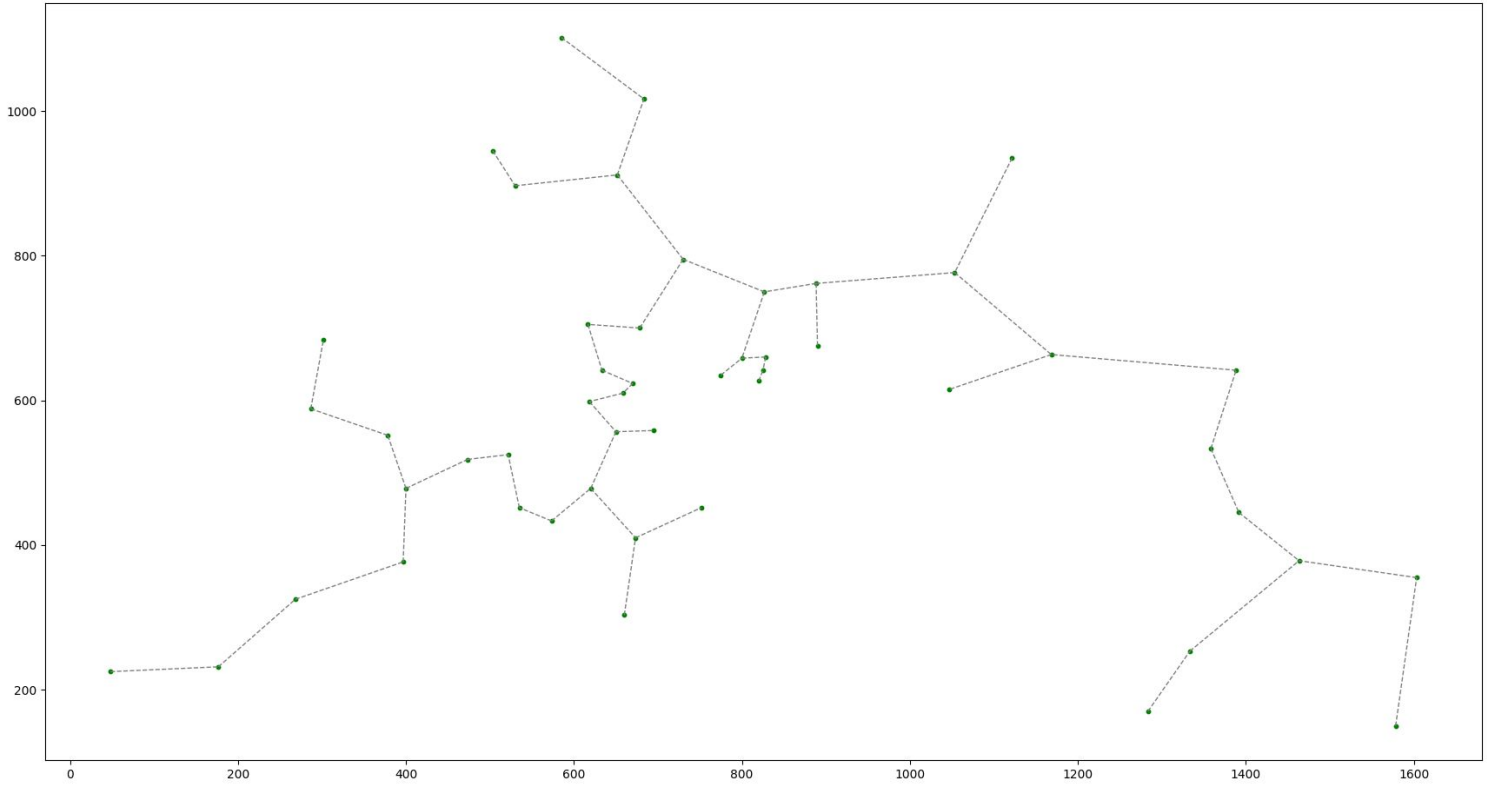
Induced Boundary (the resulting tour)



Admissible Surface with enforced **Global Connectivity**



The **Tree** ensures a single **connected surface**



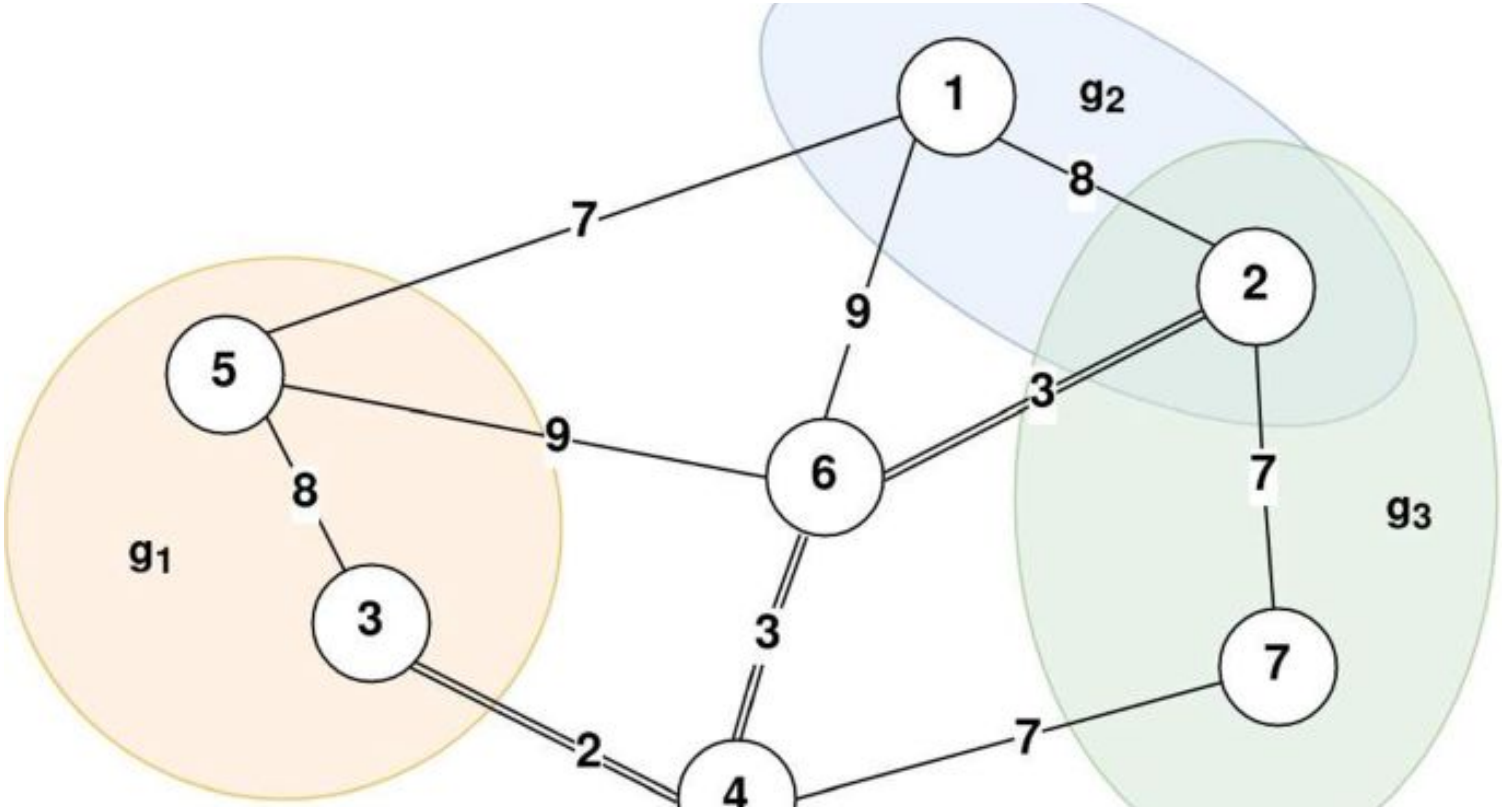
Perspective Shift: **Global Connectivity**

`circuit` of edges

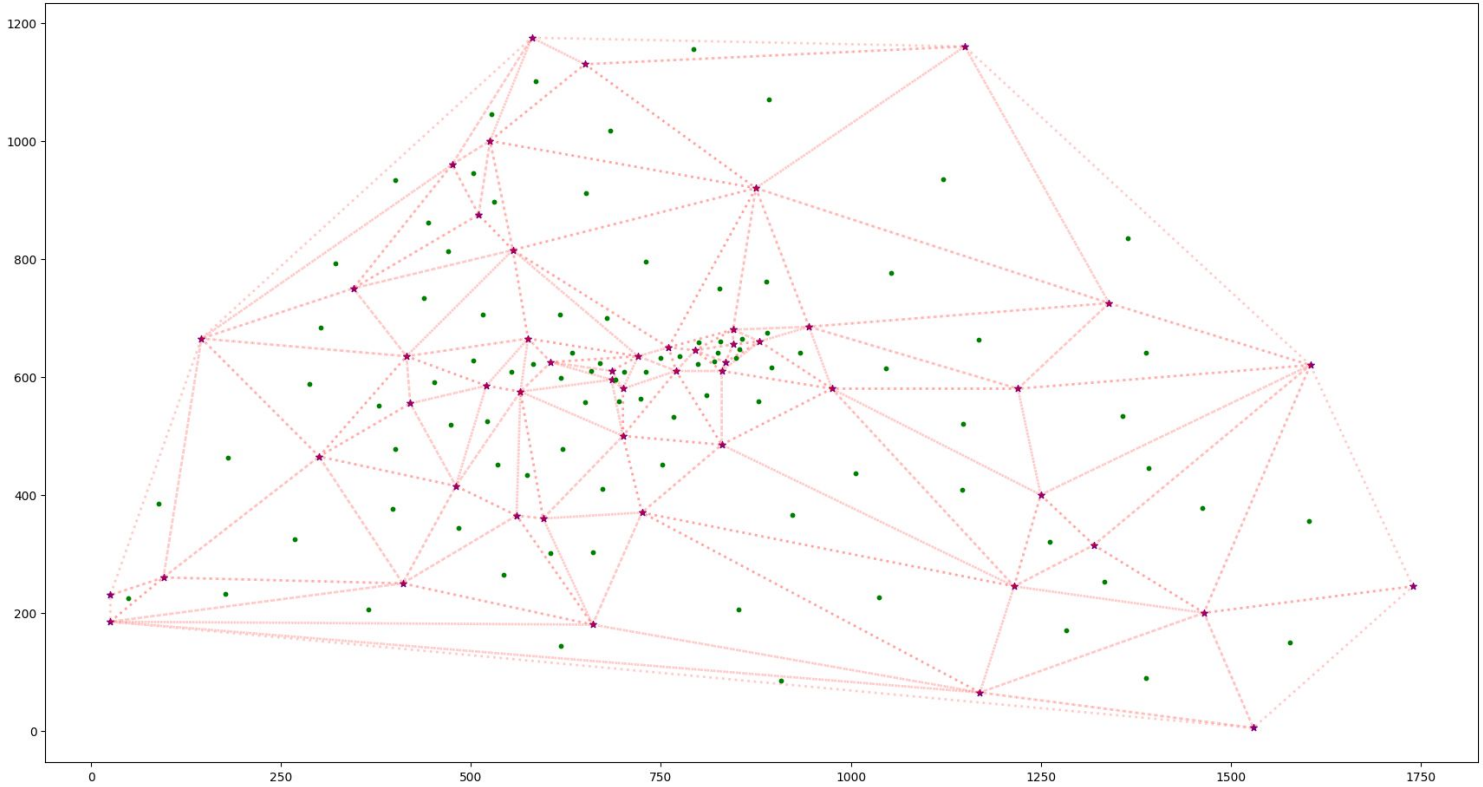
↓

`tree` of triangle nodes

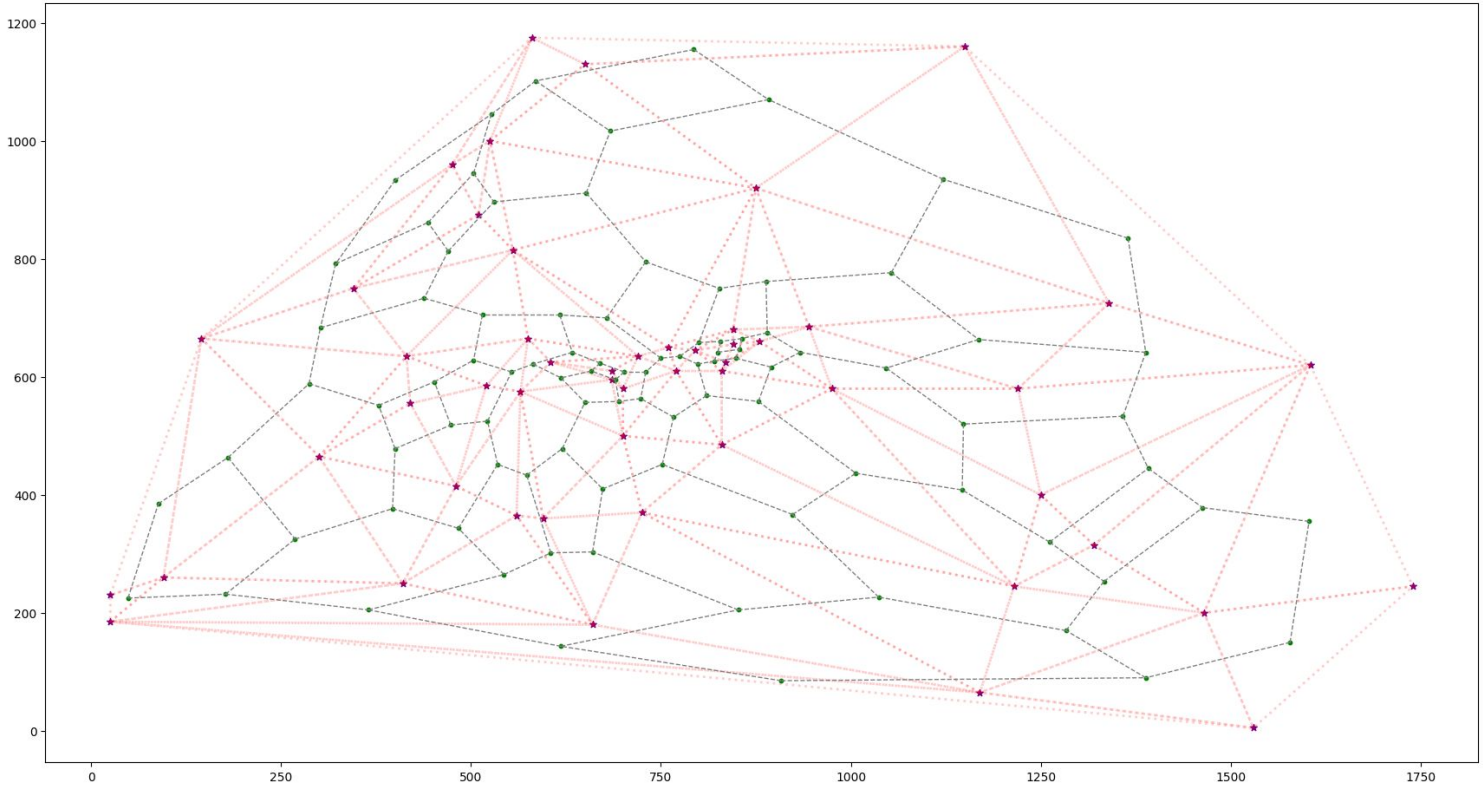
Group Steiner Tree: connect **groups** with **min. cost**



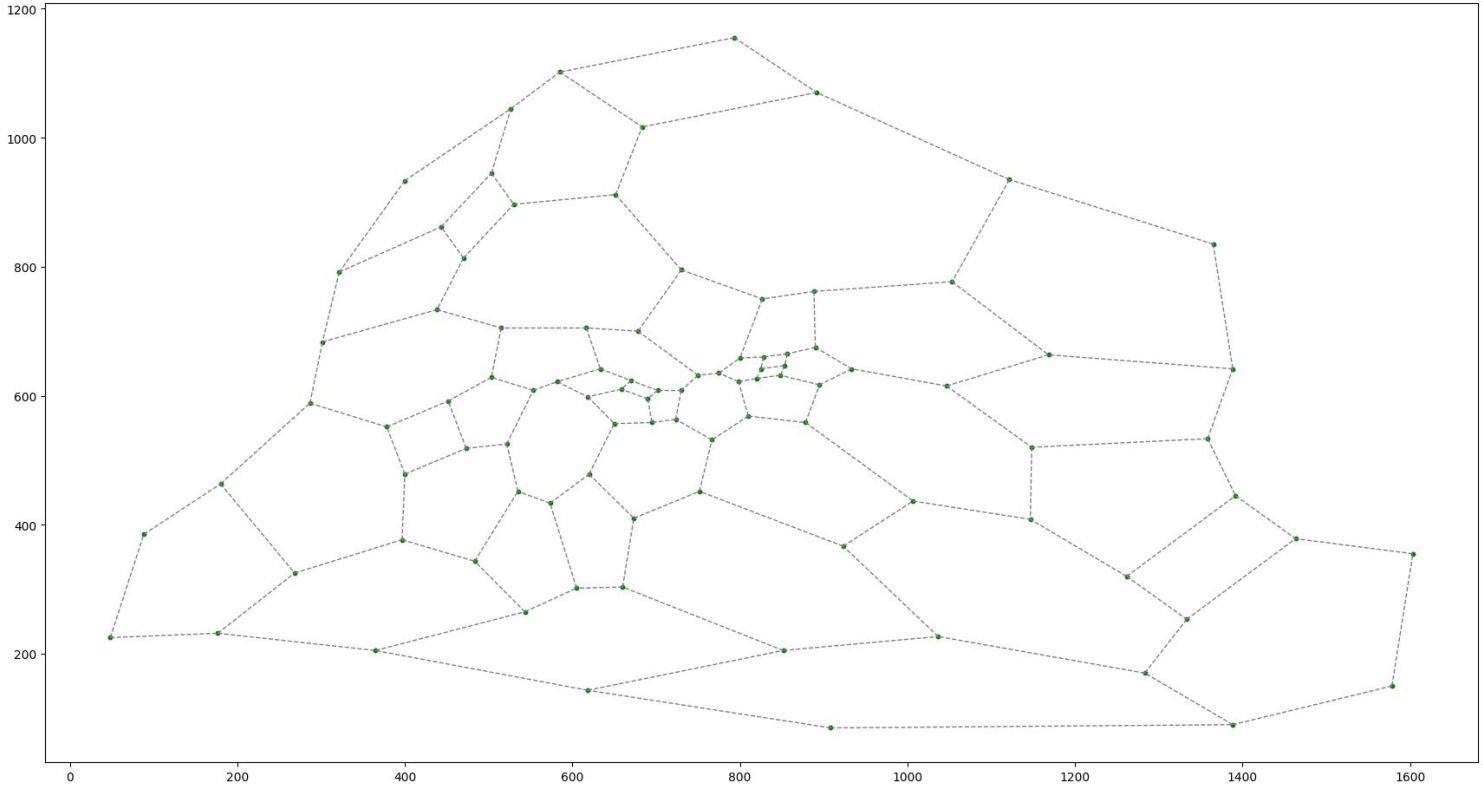
Dual Graph: **Nodes** (for each triangle)



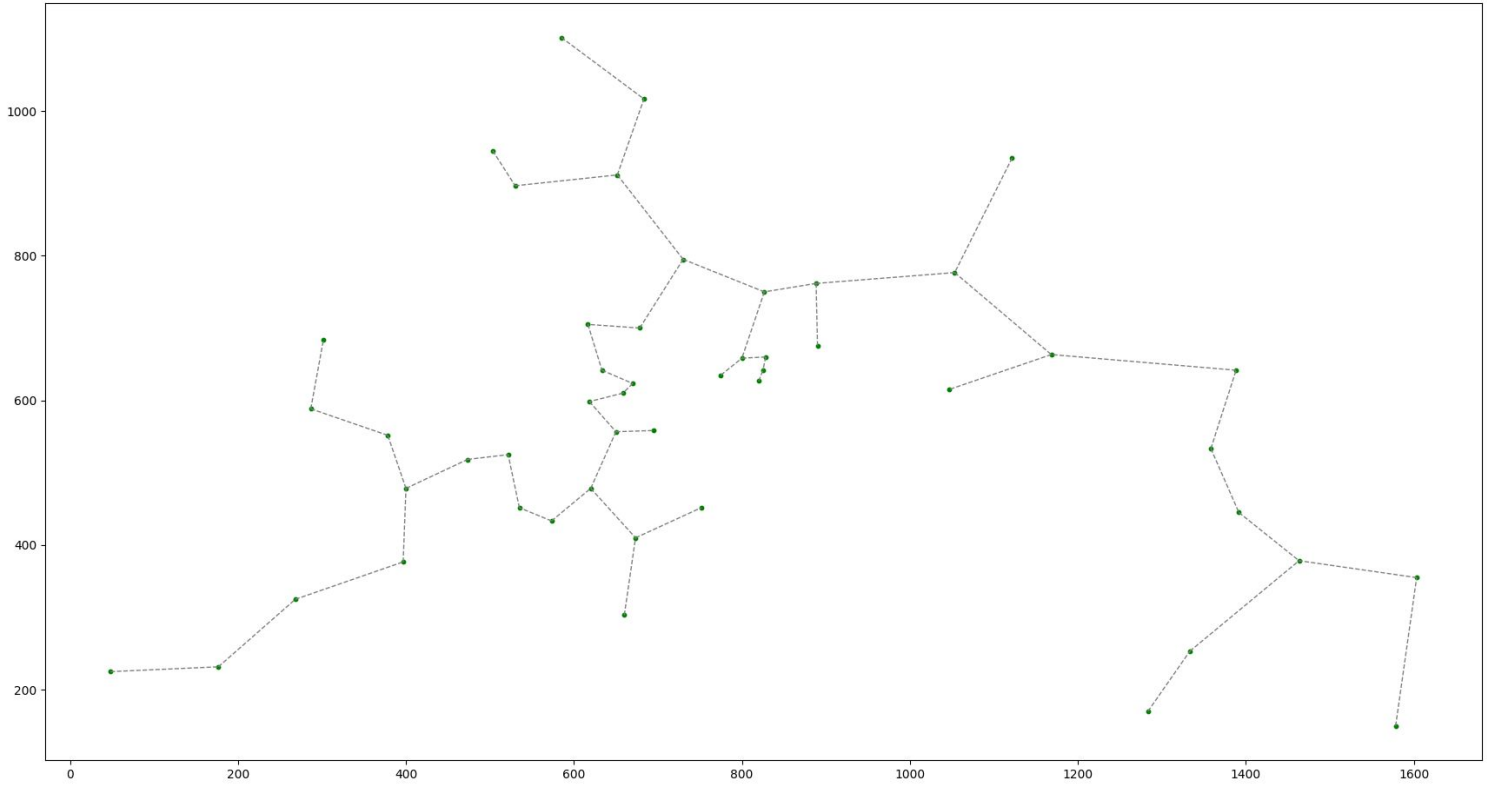
Dual Graph: **Edges** (for each shared edge)



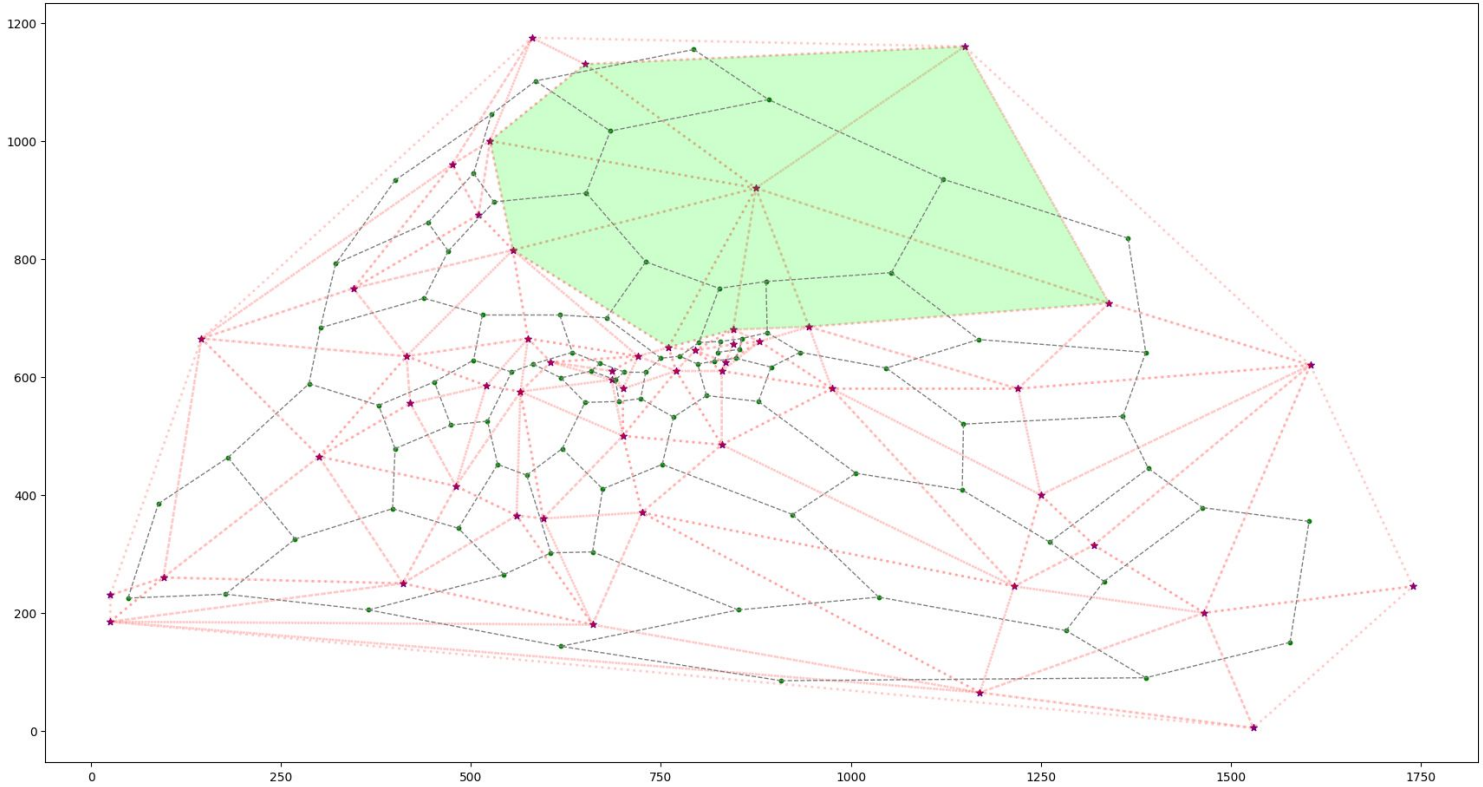
Complete **Dual Graph**



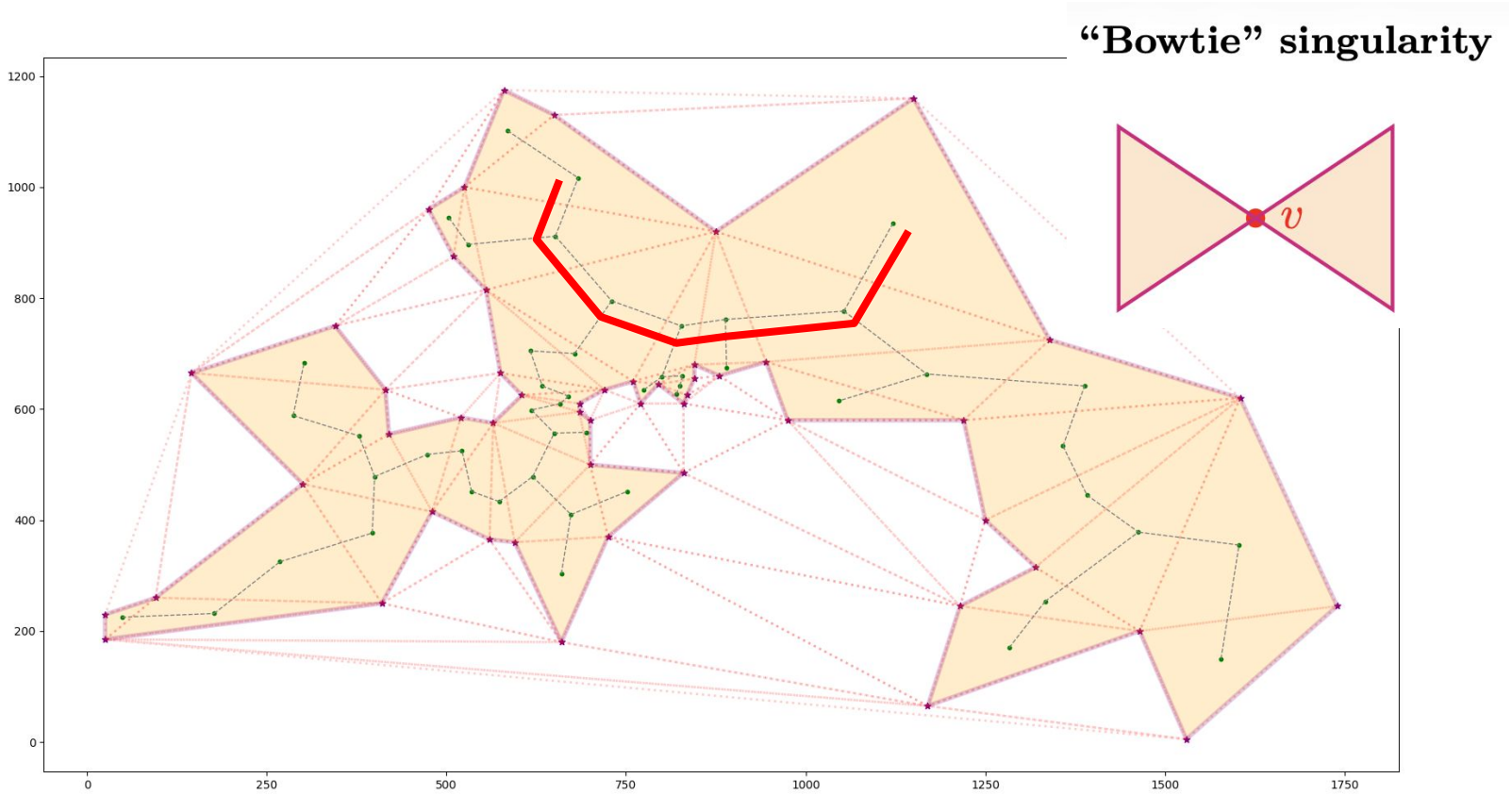
Select a **Tree** that covers **every vertex group** with **min. cost**



Group Steiner Tree: what are <u>our</u> groups?



H_v must be a **Simple Path**



Euler Characteristic

(implicitly satisfies at least one node per group)

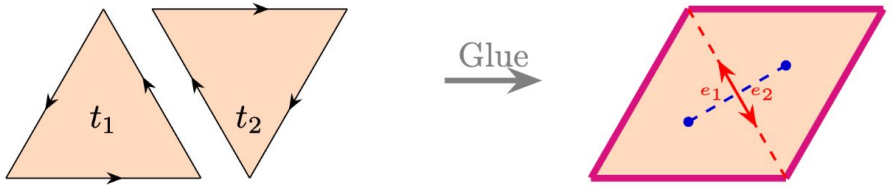
$$\chi(H_v) = |\mathrm{V}(H_v)| - |E(H_v)| = 1 \quad \forall v \in V.$$

The Cost Function (Nodes with **Cost**, Edges with **Profit**)

Boundary Cancellation. In this planar case, the objective simplifies by eliminating explicit edge-node variables, and can be derived from Eq. (2).

$$\text{Min } Z = \sum_{t \in \mathcal{T}_{selected}} P(t) - 2 \sum_{e \in \mathcal{E}_{dual}} L(e) \tag{D.1}$$

where $P(t)$ is the perimeter of triangle t and $L(e)$ is the length of the primal edge shared by the two triangles connected by dual edge e.



$-2 \times \text{Length(Internal)}$

The MiniZinc Model: **Decision Variables**

```
% Active nodes (triangles)
array[1..T] of var bool: x;

% Active edges (shared edges between triangles)
array[1..E] of var bool: y;
```

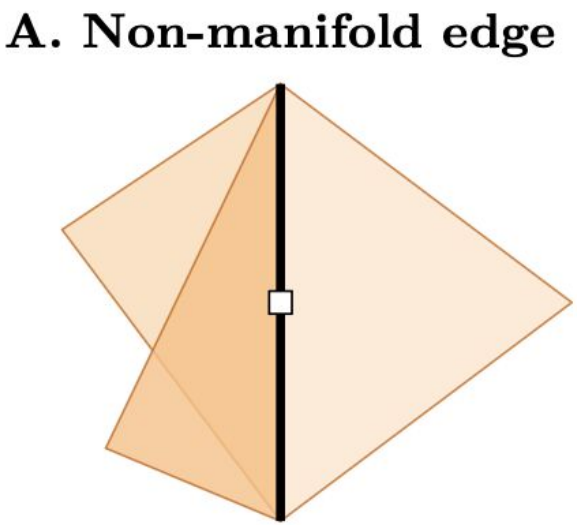
The MiniZinc Model: **Objective Function**

```
var int: objective = cost_triangle_perimeters - profit_internal_edges;

var int: cost_triangle_perimeters =
    sum(t in 1..T)(bool2int(x[t]) * triangle_perimeters[t]);

var int: profit_internal_edges =
    2 * sum(e in 1..E)(bool2int(y[e]) * dual_edge_costs[e]);
```

The MiniZinc Model: **Constraints**




```
% Dual Edge Consistency
constraint forall(e in 1..E) (
  y[e] <-> (x[u_dual_edges[e]] /\ x[v_dual_edges[e]])
);

% Degree constraints, for each primal edge
constraint forall(p in 1..P
                  where pe_triangle_incidence_length[p] > 2)(
  sum(i in 0..pe_triangle_incidence_length[p]-1)(
    bool2int(x[ pe_triangle_incidence_indices[ pe_triangle_incidence_start[p] + i ] ])
  ) <= 2
);
```

The MiniZinc Model: **Global Connectivity**

```
%%
% activate only one of the global connectivity alternatives.
% only change the include file, no need to change the constraint.
%%

include "global_connectivity/custom_tree_implementation_float.mzn";
% include "global_connectivity/custom_tree_implementation_int.mzn";
% include "global_connectivity/global_tree_native.mzn";

%%
%%

constraint global_tree_connectivity();
```

The MiniZinc Model: **Local Regularity**

```
%%
% activate only one of the local-connectivity alternatives.
% only change the include file, no need to change the constraint.
%%

include "local_connectivity/euler_filter.mzn";
% include "local_connectivity/local_tree.mzn";

%%
%%

constraint forall(v in 1..N) (
    local_connectivity(v)
);
```

Computational Results (on **Euclidean** instances)

https://github.com/hoketo/surface_based_tsp_with_minizinc

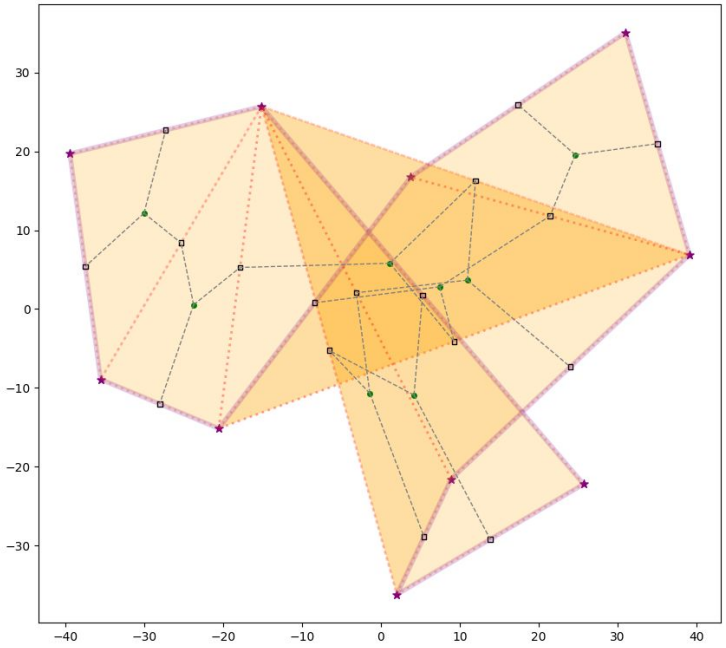
We utilized HiGHS v1.12 [19] on an Apple M4 Max. Table 2 summarizes the performance across all instances, including several from TSPLIB [26]. All experiments use the Delaunay triangulation unless stated otherwise. The results indicate that the SB-Flow generally outperforms SB-MTZ as N grows. For instances ($N \leq 280$), it typically solves at the root. For larger instances, SB-Flow yields a substantially tighter relaxation, reflected in dramatically smaller Branch-and-Bound trees (e.g., 142 nodes vs. 23,205 for `pcb442`).

Table 2 Combined Computational Results. Post-presolve MILP size and performance; values are 10-run means (unless noted), with a timeout limit of 1-hour.

		MILP Size		HiGHS	
Instance	**Model**	**Vars**	**Cons**	**Time (s) [Solved]**	**Nodes**
Berlin52	Lifted-MTZ	341	523	0.08s [10/10]	1
($N = 52$)	SB-MTZ	553	1,107	**0.02s** [10/10]	1
	SB-Flow	684	1,054	0.03s [10/10]	1
St70	Lifted-MTZ	463	723	14.23s [10/10]	2,936
($N = 70$)	SB-MTZ	759	1,521	4.35s [10/10]	257
	SB-Flow	926	1,440	**0.20s** [10/10]	**1**
Ch130	Lifted-MTZ	879	1,362	108.6s [10/10]	16,058
($N = 130$)	SB-MTZ	1,458	2,922	**0.29s** [10/10]	**1**
	SB-Flow	1,793	2,775	1.62s [10/10]	**1**
Tnm199	SB-Flow (Delaunay)	2,764	4,283	**0.51s** [10/10]	**1**
($N = 199$)	SB-Flow (Greedy)	2,090	3,202	4.76s [10/10]	1.6
A280	Lifted-MTZ	1,907	2,992	DNF [0/5]	>53k
($N = 280$)	SB-MTZ	3,188	6,379	29.0s [10/10]	490
	SB-Flow	3,923	6,056	**5.68s** [10/10]	**1**
Pcb442	SB-MTZ	5,195	10,393	1656s* [7/10]	23,205
($N = 442$)	SB-Flow	6,412	9,879	**81.1s** [10/10]	**142**
d657	SB-Flow	9,634	14,837	TLE (0.60%)	>30k
($N = 657$)					

* For `pcb442` SB-MTZ, 3/10 runs exceeded the 1-hour time limit; time and nodes are averaged over the 7 solved runs.

Generalization: **non-metric**



Generalization: the approach

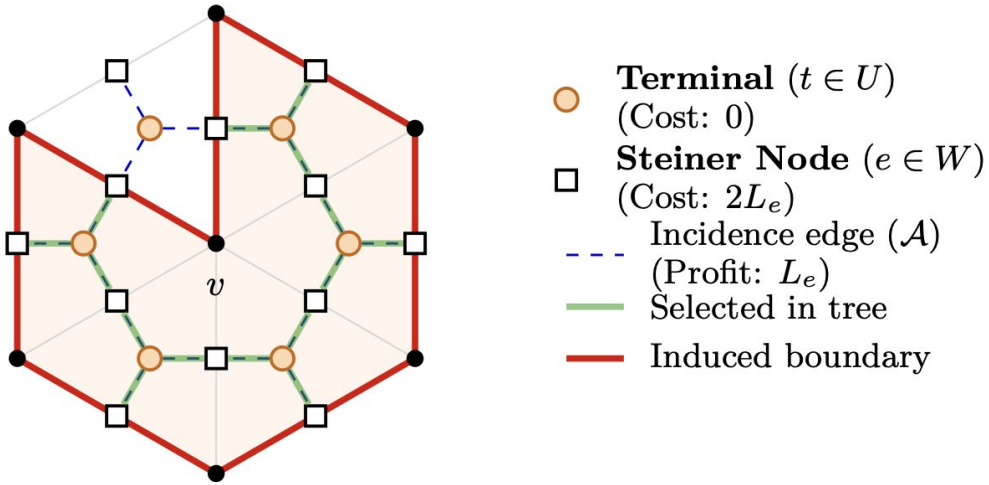


Figure 3 Bipartite incidence graph: triangles x_t (circles) and edges y_e (squares); dashed lines are adjacencies. A feasible solution selects a connected set of triangles (filled orange), inducing the highlighted boundary (red) around the center; translucent green edges indicate incidence edges chosen in the resulting tree.

Conclusion

- Exact **equivalence** when T = C(V, 3) despite being computationally intractable
- **Sparse candidate sets** (e.g. Delaunay, Greedy triangulations) yield **practical heuristics**
- A tree constraint **replaces subtour elimination** conceptually

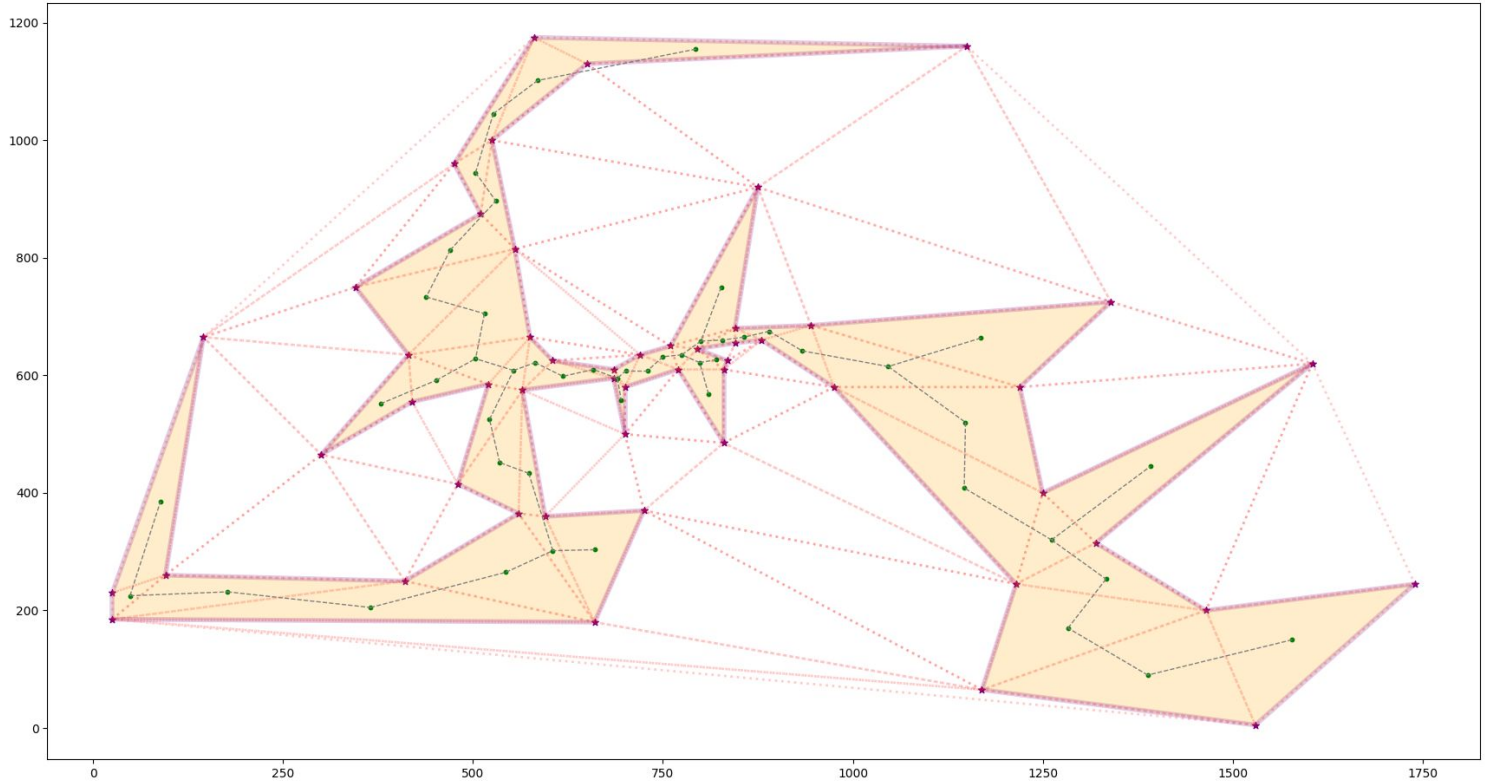
Advantages

- Looking at the problem from a **geometric-topological perspective**,
- Interior is explicit, so **geometric constraints** become more natural,
- Suggests new links to **Steiner-tree-type methods**.

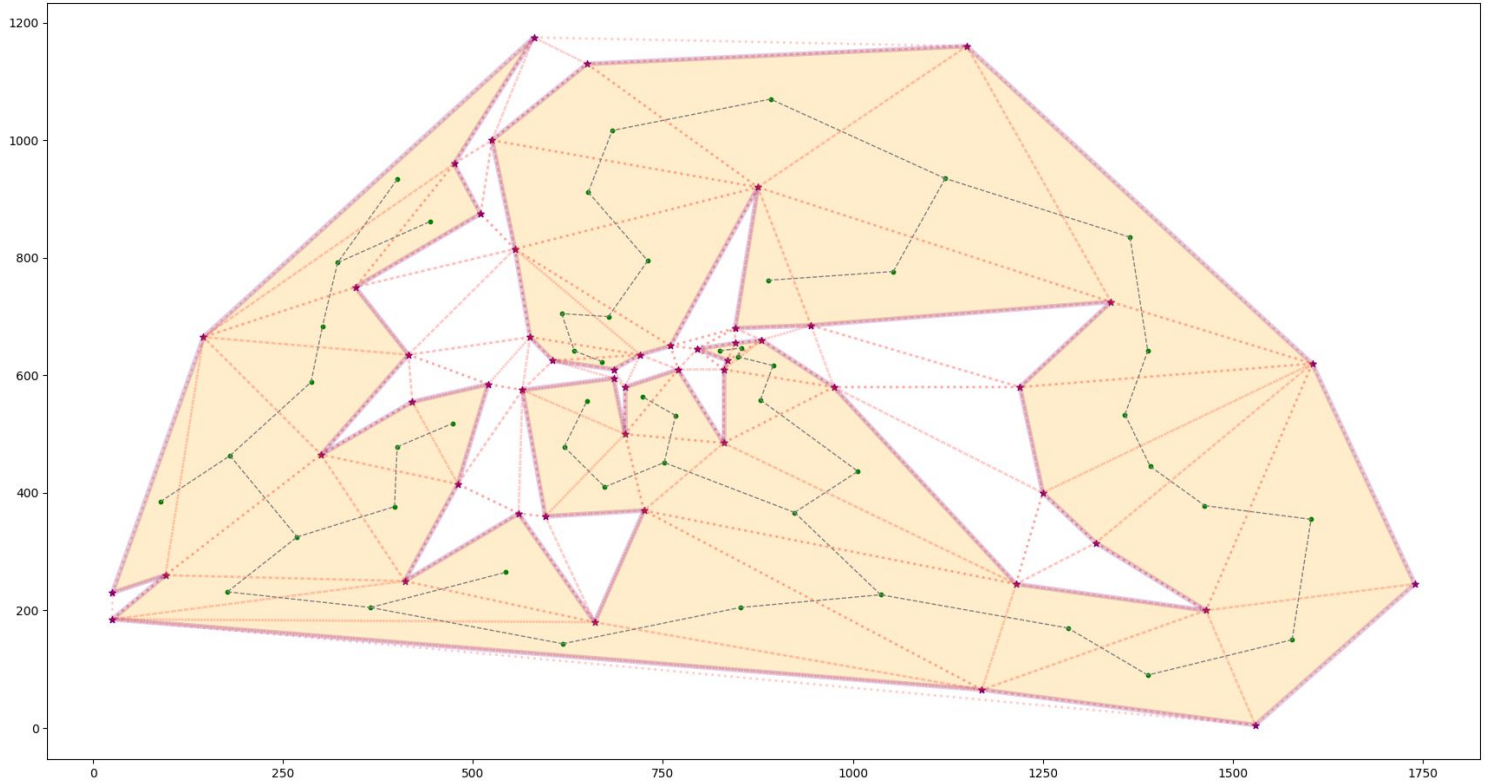
Disadvantages

- Triangulations may exclude Hamiltonian cycles,
- Computational behavior depends strongly on candidate complex choice and remains open for further study,
- Tightness on triangulations is not yet theoretically understood.

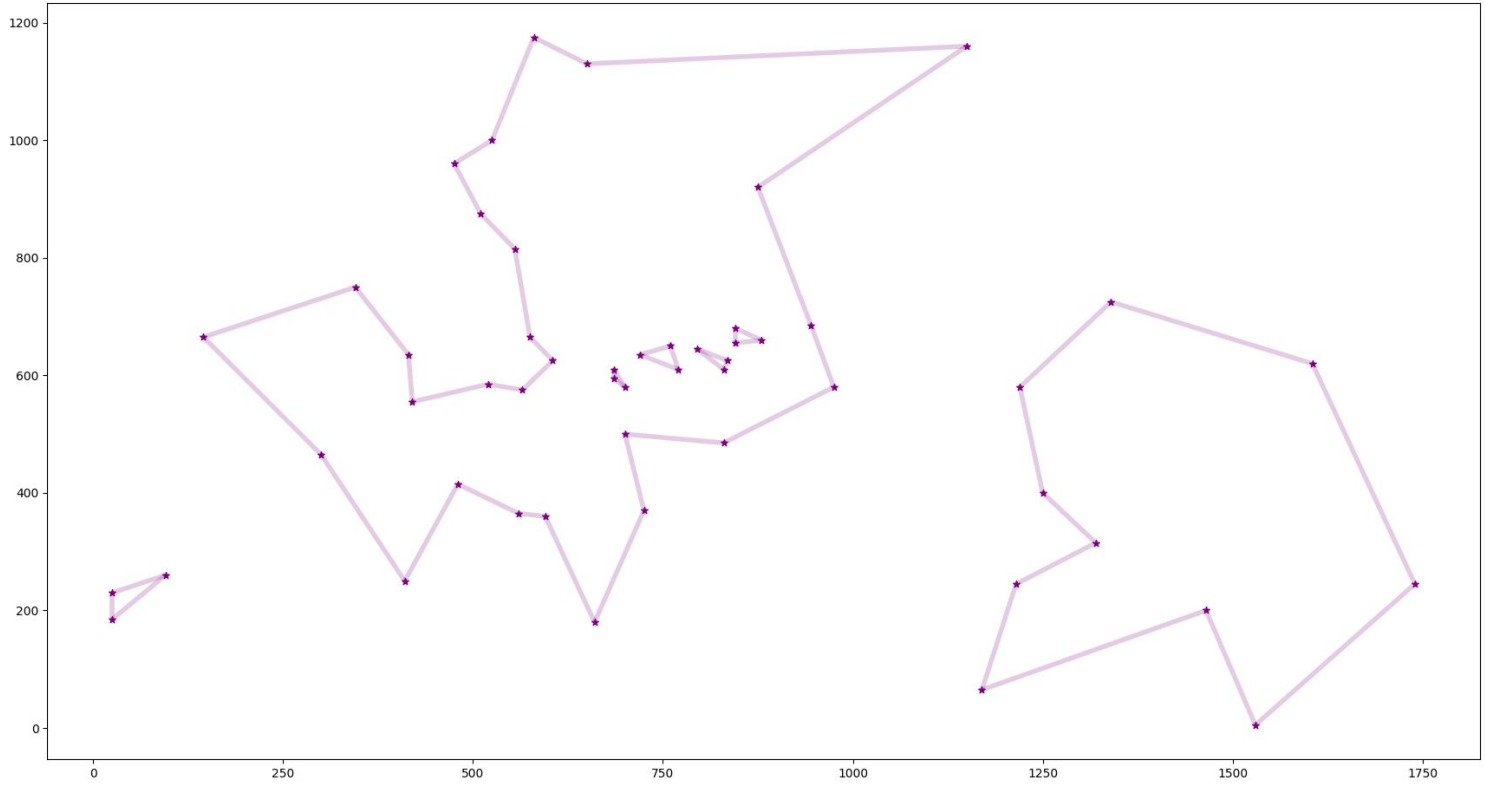
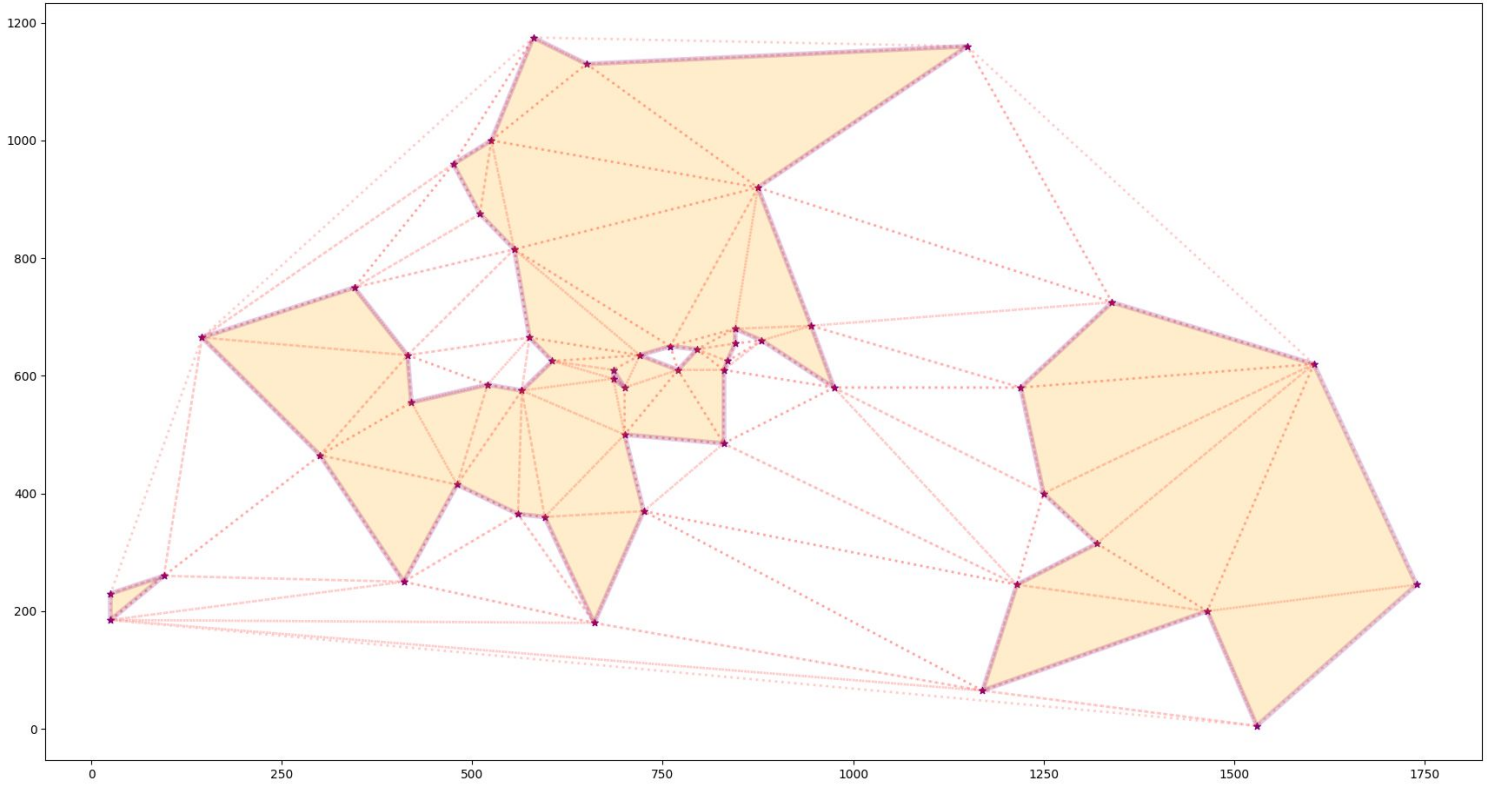
Minimized Area (Node **Cost** is the **Area** of the triangle)



Maximized Area (Node **Profit** is the **Area** of the triangle)



Subtours (Local Regularity: node **degree = 2**)



Local Regularity Around Each City